

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

BOIDS IN UNITY

Performance Scaling and GPU Spatial Grid Analysis

Matt Delalieux (r0890338)
Radi Miah (r1012080)

Academic year 2025–2026

Contents

1 Introduction **2**
- 1.1 Problem Context . . . 2
- 1.2 Project Goal . . . 2

2 Architectural Design: CPU Pipeline **2**

3 Architectural Design: GPU Pipeline **3**
- 3.1 Core Buffer Architecture . . . 3
- 3.2 Phase 1: Initialization & Counting (`ClearGrid`, `CountGrid`) . . . 3
- 3.3 Phase 2: The Prefix Sum (Blelloch Scan) . . . 3
- 3.4 Phase 3: The Offset Copy (`CopyOffsets`) . . . 3
- 3.5 Phase 4: Memory Scatter & Sorting (`ScatterBoids`) . . . 4
- 3.6 Phase 5: Physics Update (`UpdateBoids`) . . . 4

4 Performance Analysis **5**
- 4.1 The CPU Degradation Curve . . . 5
- 4.2 The GPU Scalability Plateau . . . 5
- 4.3 GPU Memory Bandwidth Saturation . . . 6
- 4.4 The Hard Drop-Off: TDR Driver Reset . . . 6

5 Exploring GPU Concepts via Unity **6**
- 5.1 Global Memory and Coalesced Access . . . 6
- 5.2 Race Conditions and Hardware Atomics . . . 7
- 5.3 Shared Memory Limits . . . 7
- 5.4 Warps and Thread Divergence . . . 7
- 5.5 Asynchronous Execution . . . 7

6 Conclusion **8**

1 Introduction

1.1 Problem Context

A Boids simulation mimics flocking behavior by evaluating three rules for every boid: cohesion, alignment, and separation. This simulation requires per-frame position updates, neighbourhood queries, spatial hashing, and rendering. Low-level GPU programming (CUDA) exposes explicit control over memory hierarchy, thread scheduling, and concurrency. Unity's Compute Shader and C# provides a higher-level abstraction but constrains certain optimisations.

1.2 Project Goal

The primary goal of this project is to implement a fully GPU-driven Boids simulation in Unity and compare its performance and architectural design against a CPU-bound pipeline. By offloading the entire spatial hashing and simulation logic to the GPU, we aim to demonstrate the scalability and efficiency gains achieved by eliminating CPU-side loops and memory bottlenecks.

As a secondary objective, this report uses our new GPU pipeline as a case study to explore the low-level GPU programming concepts learned in the labs (coalesced memory access, race conditions, thread divergence, and asynchronous execution). By mapping these raw CUDA concepts onto Unity's Compute Shader system, a comparison can be made to highlight where Unity provides productivity gains and where its high-level abstractions introduce performance limitations.

2 Architectural Design: CPU Pipeline

In the CPU architecture, the processor is entirely responsible for generating the spatial grid and resolving the flocking behaviors. The simulation state is maintained in memory using `NativeArray<Boid>`. Every frame, the CPU must sequentially execute the following steps:

1. **Clear Grid:** Iterate through the cell arrays to reset counters to zero.
2. **Hash and Count:** Iterate through every boid to calculate its spatial cell based on its 2D coordinates and increment the corresponding cell counter.
3. **Generate Offsets (Prefix Sum):** Calculate a running total of the cell counts to determine the starting index for each cell in a flat 1D array. For example, if Cell 0 has 5 boids and Cell 1 has 3, Cell 1's data starts at index 5, and Cell 2's data starts at index 8.
4. **Rearrange Boids (Scatter):** Iterate through the boids again. Each boid reads its cell's starting index, writes itself into that slot in a new array, and increments the index. This physically packs boids that share a grid cell into contiguous memory.
5. **Update Behaviors:** Iterate through the sorted boids, query neighboring cells, apply the flocking rules, limit maximum speed, and restrict them to screen bounds.

Once the CPU finishes this serial computation, the updated boid positions must be rendered. Because rendering occurs on the GPU, the CPU must upload the entire `NativeArray` array to the GPU's Video RAM (VRAM). At large scale, this mandatory host-to-device memory transfer creates a latency bottleneck.

3 Architectural Design: GPU Pipeline

3.1 Core Buffer Architecture

To understand the pipeline's mechanics, it is essential to divide its data structures into two distinct categories: **Grid Metadata** (which tracks array indices) and **Boid Data** (which stores the physical simulation state).

- `gridOffsetsRWBuffer` **(Grid Metadata - Mutable):** A buffer of integers keeping track of the *current insertion index* for each grid cell during the sorting phase.
- `gridOffsetBuffer` **(Grid Metadata - Read-Only):** A frozen backup of the calculated cell starting indices, preserved before any sorting mutates them.
- `boidBufferOut` **(Boid Data - Sorted Input):** A buffer of `Boid` structs holding the physically sorted entities (grouped by grid cell). This acts as the heavily-read input state for the physics step.
- `boidBuffer` **(Boid Data - Primary Output):** A buffer of `Boid` structs acting as the destination for the newly calculated physics state. This is also the buffer bound to the Rasterizer to draw the boids on the screen.

3.2 Phase 1: Initialization & Counting (`ClearGrid`, `CountGrid`)

The pipeline begins by dispatching `ClearGrid` (1 thread per grid cell) to zero out the `gridCounts` and `gridOffsetsRW` buffers. Next, `CountGrid` is dispatched (1 thread per boid). Each boid determines its 2D grid cell index based on its spatial coordinates. It then executes an atomic `InterlockedAdd` on the `gridCounts` buffer to safely increment the population count of that specific cell.

3.3 Phase 2: The Prefix Sum (Blelloch Scan)

Knowing how many boids reside in each cell is insufficient; the GPU needs to know the exact global memory index where each cell's data should begin in the flat array. The starting indices are calculated by performing an exclusive Prefix Sum (Blelloch Scan) over the `gridCounts` array. Because Unity limits thread synchronization to a single block (max 1024 threads), a global prefix sum across a massive grid requires three distinct dispatch passes:

- `PrefixSumBlocks`**:** This pass calculates the prefix sum locally for individual blocks of 1024 cells. Each thread block loads its portion of the `gridCounts` array into groupshared memory, an on-chip SRAM region private to each SM. Avoiding the latency of repeated global memory reads. It performs a parallel up-sweep (reduction) and down-sweep to compute the local offsets. The total sum of all boids within this 1024-cell block is recorded and written to a separate buffer (`gridGroupSums`), representing the total population of this specific block.
- `PrefixSumGroupTotals`**:** This pass takes the block totals generated in the previous step and performs a second prefix sum exclusively on them. By doing so, it determines the absolute global starting index for every block. For example, if Block 0 contains 5,000 boids in total, Block 1's global offset must start at index 5,000.
- `ApplyGroupTotals`**:** This final pass distributes the global block offsets back to the individual cells. It adds the global block offset (from Step 2) to the local cell offset (from Step 1). The final result is written to `gridOffsetsRWBuffer`, providing the exact, ready-to-use global memory index where each cell's data block begins.

3.4 Phase 3: The Offset Copy (`CopyOffsets`)

Before sorting the boids into their newly calculated memory slots, the `gridOffsetsRWBuffer` must be backed up into the read-only `gridOffsetBuffer`. This copy is mandatory because the upcoming sorting phase physically destroys the original offsets. As boids are inserted into memory, the offset index is atomically incremented by 1. By the time all boids are sorted, `gridOffsetsRWBuffer` no longer points to the *start* of the grid cells, it points to the *end* of them. The `CopyOffsets` pass preserves the starting indices so the final physics phase can actually locate neighboring cells.

3.5 Phase 4: Memory Scatter & Sorting (`ScatterBoids`)

The `ScatterBoids` kernel is dispatched with one thread per boid. Each thread reads its boid from the unsorted input, recalculates the boid's cell, and executes an atomic `InterlockedAdd` against the mutable `gridOffsetsRWBuffer[cell]`.

This atomic operation simultaneously retrieves a unique insertion index and increments the cell's counter for the next thread. The metadata buffer only dictates *where* data should go, the thread must now physically move the `Boid` struct into the `boidBufferOut` array at that retrieved index. This physical sorting guarantees that boids sharing a grid cell are contiguous in 1D GPU memory. Because horizontally adjacent cells are also contiguous in the 1D array, the entire three-cell width of each row in the 3×3 neighbourhood can be read as a single contiguous memory range. The two vertical jumps between rows are non-contiguous but predictable and small in number. This enables substantially more coalesced memory access during the physics step compared to an unsorted buffer.

3.6 Phase 5: Physics Update (`UpdateBoids`)

With the data physically sorted in global memory, the physics simulation executes. The `UpdateBoids` kernel calculates cohesion, alignment, and separation by iterating over neighboring grid cells. Using the preserved `gridOffsetBuffer`, finding the start and end indices of a neighbouring cell is reduced to an $O(1)$ lookup (Listing 1). The loop bounds are clamped per-frame using the boid's own grid coordinates, ensuring that boids sitting on the grid boundary do not generate out-of-bounds buffer accesses. This produces a true 3×3 neighbourhood search across all three row offsets and all three column offsets.

This phase relies strictly on a Double-Buffering technique to prevent Read-After-Write (RAW) data races. If only a single buffer were used, Thread A might update its position in memory just a microsecond before Thread B attempts to read Thread A's position to calculate cohesion. Thread B would inadvertently read future state instead of current state, leading non-deterministic physics.

To solve this, all threads safely *read* the frozen, sorted state of the previous frame from `boidBufferOut`, calculate their new velocities and positions, and *write* the updated state to the primary `boidBuffer`. Because no thread reads from `boidBuffer` during this step, data races are entirely prevented. Once the pass completes, `boidBuffer` holds the perfect, up-to-date state of the flock, ready to be consumed by the Rasterizer for rendering.

```
// Clamp row range to avoid out-of-bounds access at grid edges
int rowStart = max(-1, -(int)gridXY.y);
int rowEnd   = min( 1,  (int)(gridDimY - 1 - gridXY.y));

for (int i = rowStart; i <= rowEnd; i++) {
    uint y = cell + i * gridDimX;

    // Clamp column range and find start/end indices in the sorted array
    uint xStart = (gridXY.x > 0)            ? y - 1 : y;
    uint xEnd   = (gridXY.x < gridDimX - 1) ? y + 2 : y + 1;

    uint start = gridOffsetBuffer[xStart];
    uint end   = gridOffsetBuffer[xEnd];

    for (uint j = start; j < end; j++) {
        Boid other = boidsIn[j]; // Reads safely from boidBufferOut
        float2 diff = boid.pos - other.pos;
        float distSq = dot(diff, diff);

        if (distSq > 0 && distSq < visualRangeSq) {
            // Apply Cohesion, Alignment, Separation physics...
        }
    }
}
```

Listing 1: O(1) 3×3 neighbour lookup with boundary clamping.

4 Performance Analysis

To objectively quantify the efficiency gains for offloading the spatial data structures and physics calculations to the GPU, the Average Frames Per Second (FPS) of both the CPU and GPU architectures were benchmarked. The entity count was scaled exponentially from 32 boids up to over 2.4 million boids.

The benchmarks were executed on a mobile hardware configuration featuring an Intel® Core™ i7-8565U CPU (1.80GHz) and an NVIDIA GeForce MX150 discrete GPU. While these are low-power, entry-level mobile components, they provide an excellent environment for exposing architectural bottlenecks without brute-forcing through them. The results are visualized in Figure 1.

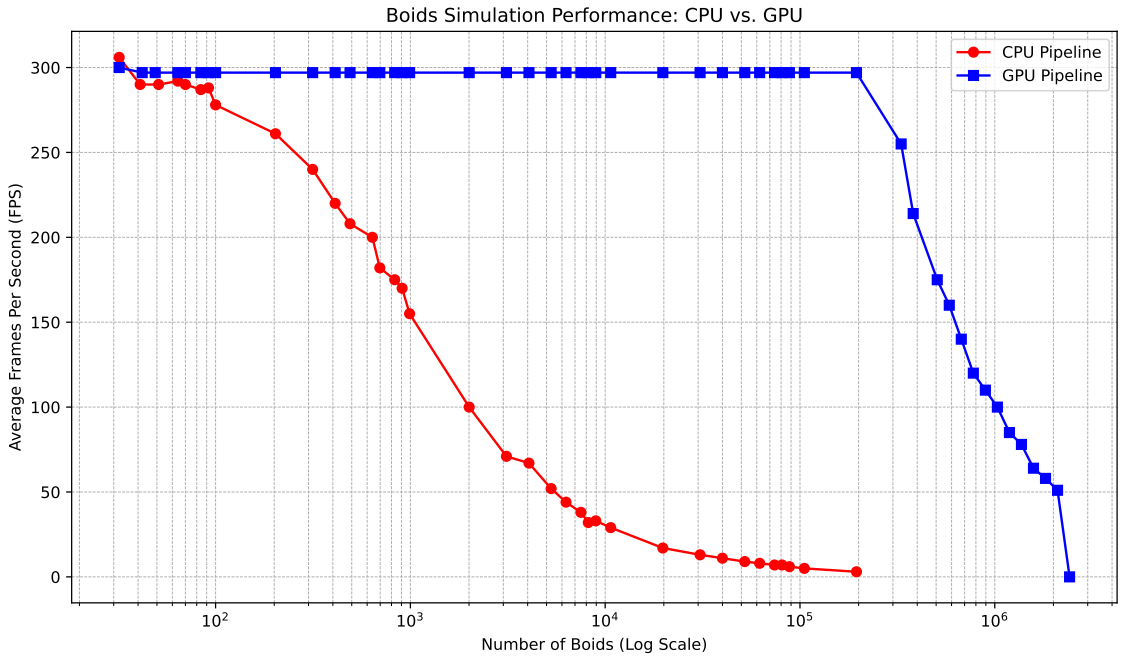


Figure 1: Performance scaling comparison between the CPU and GPU pipelines on an Intel i7-8565U and NVIDIA MX150. Note the logarithmic scale on the X-axis.

4.1 The CPU Degradation Curve

The strict limitations of the CPU-bound pipeline becomes immediately apparent. Because the host processor (i7-8565U) must sequentially iterate through every single boid to calculate grid hashes, perform memory scatters, and execute $O(N_{neighbors})$ distance checks, performance scales poorly, with the compute time growing roughly linearly ($O(N)$) as the data pool expands. Because framerate is the inverse of frame time, this linear cost growth is seen as a sharp $1/N$ plunge in performance.

The CPU pipeline fails to maintain a standard 60 FPS threshold extremely early, dropping to 52 FPS at just 5,279 boids. Past 10,000 entities, the CPU cache is likely thrashing heavily due to the sheer volume of scattered memory lookups required for the neighborhood searches. The mandatory `SetData()` upload, moving the entire `NativeArray` to VRAM every frame for rendering, adds a compounding bottleneck as boid count grows. By 30,703 boids, the simulation runs at 13 FPS, and at 105,297 boids, it reaches an unplayable 5 FPS.

4.2 The GPU Scalability Plateau

The fully GPU-driven pipeline scales considerably better, particularly given the constraints of the MX150 (a chip with only 384 CUDA cores). Because the simulation data never leaves the GPU's VRAM, PCI-e transfer latency is mostly eliminated.

As seen in Figure 1, the GPU easily processes the workload and remains bottlenecked by the Unity engine's internal framerate cap ($\approx$ 297 FPS) all the way up to roughly 100,000 boids. During this plateau, the GPU

is underutilized, the SMs process the relatively small warps so quickly that frame dispatching overhead is the primary limiting factor.

4.3 GPU Memory Bandwidth Saturation

The GPU only begins to show natural performance degradation at 330,691 boids (255 FPS). From this point forward it shows a stable, linear degradation curve on the logarithmic graph.

This steady scaling, rather than a sudden collapse, is a direct result of the Phase 4 `ScatterBoids` memory sorting. Because the boids are physically packed into contiguous memory blocks by grid cell, memory reads during the `UpdateBoids` neighbour search are coalesced, reducing the number of cache line transactions per warp.

However, as the simulation passes 1 million entities, the hardware limits of the MX150 begin to bottleneck the pipeline. Doubling the visual range (while keeping the spatial grid fixed) reduced frame rate by 50%, demonstrating that the simulation is limited by neighbour-processing throughput rather than peak memory bandwidth. The GPU spends most of its time on arithmetic and branching inside the neighbour loop, not waiting for data from VRAM. Despite this, the GPU gracefully sustains playable, real-time framerates (64 FPS) pushing 1,580,399 boids simultaneously.

4.4 The Hard Drop-Off: TDR Driver Reset

At approximately 2.42 million boids, the framerate abruptly collapses to 0 FPS. This is not caused by running out of Video RAM (capacity), but rather by an execution time hardware timeout.

When attempting to dispatch the simulation at 2.4 million entities, the Unity Editor forcefully shuts down, throwing the following unrecoverable graphics API error:

> *"Failed to present D3D11 swapchain due to device reset/removed. This error can happen if you draw or dispatch very expensive workloads to the GPU, which can cause Windows to detect a GPU Timeout and reset the device."*

This is a **Timeout Detection and Recovery (TDR)** event enforced by Windows. To prevent frozen graphics cards from locking up the entire computer, Windows monitors how long the GPU takes to complete a single task. By default, if a GPU takes longer than 2 seconds to complete a dispatch, the OS assumes the GPU is caught in an infinite loop and forcefully preempts it by killing and resetting the graphics driver.

Because the MX150 only has 384 CUDA cores, processing 2.42 million boids requires executing over 75,000 warps per kernel. At this scale, the load of the `UpdateBoids` compute shader simply takes longer than 2 seconds to finish computing on the SMs. Windows detects this delay, intercepts the execution, resets the D3D11 device, and crashes the engine.

Ultimately, this data proves that migrating spatial hashing logic to the GPU completely shifts the bottleneck away from CPU processing and PCI-e transfer limitations, and pushes the simulation capacity to the absolute physical limits of the GPU's memory bandwidth and OS-level execution timeouts.

5 Exploring GPU Concepts via Unity

Having established the mechanics and performance of the GPU pipeline, the low-level raw CUDA concepts explored in the labs can be directly mapped to the Unity Compute Shader implementation.

5.1 Global Memory and Coalesced Access

In a raw CUDA environment, programmers explicitly manage device global memory via `cudaMalloc` and `cudaMemcpy`. Unity abstracts this entirely through the `ComputeBuffer` class. In our pipeline, buffers are allocated persistently, and the data remains exclusively in global memory.

A critical concept for GPU optimization is **memory coalescing**. When a warp of 32 threads executes a read instruction, the GPU hardware attempts to combine these requests into a minimal number of 128-byte cache line transactions. If threads request scattered memory addresses, the hardware must perform multiple distinct memory fetches, lowering memory bandwidth.

The `ScatterBoids` kernel sorts boids by grid cell, so that boids close to each other in 2D space end up consecutive in 1D global memory. This sorting has no effect on coalescing by itself. The benefit is realised in `UpdateBoids`, where adjacent threads within a warp query the same neighbourhood and read from contiguous addresses, allowing the hardware to satisfy multiple thread requests in a single 128-byte cache line transaction.

5.2 Race Conditions and Hardware Atomics

Generating a spatial grid in parallel inherently introduces race conditions. In the `CountGrid` kernel, multiple threads will inevitably attempt to write to the exact same cell counter at the exact same moment. If standard arithmetic addition were used, data races would result in lost counts.

Unity provides hardware-level atomics via `InterlockedAdd`, identical in mechanics to CUDA's `atomicAdd`. As utilized in our `ScatterBoids` pass, `InterlockedAdd` increments an array offset and returns the original value *before* the addition occurs. This guarantees that every thread is assigned a unique memory insertion index, completely preventing write-collisions during the memory scatter phase. Notably, Unity does not support floating-point atomics. If global reductions on boid velocity vectors were required, a custom integer-encoding workaround would be necessary.

5.3 Shared Memory Limits

Unity grants explicit access to Shared Memory[1] via the `groupshared` keyword, allowing us to implement the fast up-sweep and down-sweep phases of the Blelloch scan entirely on-chip.

However, because `GroupMemoryBarrierWithGroupSync()` only synchronizes threads within a single block, a global prefix sum requires three separate kernel dispatches, each round-tripping through the CPU. In raw CUDA, libraries such as `cub::DeviceScan` encapsulate this multi-pass complexity behind a single host-side call, keeping the inter-pass coordination on-device. Unity exposes no equivalent abstraction, making the CPU round-trips explicit and unavoidable in our implementation.

5.4 Warps and Thread Divergence

GPUs process instructions in warps (groups of 32 threads). If threads within a warp execute a `for` loop a different number of times, the warp diverges, and the hardware serializes execution.

In the `UpdateBoids` neighbour search, divergence arises because threads iterate different numbers of times depending on local boid density (*loop-count divergence*). Without any ordering, a warp may simultaneously contain threads whose boids are scattered across the simulation space. Thread 0 might query a dense cluster (looping 50 times) while Thread 1 queries an empty region (looping 0 times), causing severe serialization as threads wait for their neighbours to finish.

Our memory sorting (Scatter pass) mitigates this by increasing the *probability* that threads within the same warp process spatially nearby boids. Because the buffer is sorted by grid cell, threads that are consecutive in the dispatch are more likely to query grid cells of similar occupancy, reducing variance in loop iteration counts across the warp. This does not eliminate divergence entirely: boids near cell boundaries still search into adjacent cells with potentially different densities, and the 1D sort order does not guarantee 2D spatial locality. However, it substantially reduces the worst-case scenario of a single warp spanning boids from entirely different regions of the simulation.

5.5 Asynchronous Execution

In raw CUDA, asynchronous execution between the CPU and GPU is manually orchestrated via CUDA Streams. Unity abstracts this complexity entirely into an implicit command buffer system.

During the `Update()` loop, the C# script triggers all ten `Dispatch` calls sequentially (eight for the simulation pipeline, followed by two for the rasterizer). Because `GetData()` is never called (which would force the CPU to block and wait for the GPU), the dispatch commands are "fire-and-forget". The CPU instantly proceeds to the next frame logic, processing user inputs and camera movements entirely in parallel while the GPU handles the physics simulation. While this provides excellent CPU/GPU overlapping out of the box, it lacks the fine-grained control of CUDA streams required to explicitly overlap distinct compute kernels simultaneously on different SMs.

[1] The same SRAM block can be dynamically partitioned between L1 cache and shared memory.

6 Conclusion

This project demonstrates the performance advantage of a fully GPU-driven pipeline over a CPU-bound one for large-scale flocking simulations. By migrating spatial grid construction, array sorting, and physics evaluation entirely to VRAM using an 8-pass simulation pipeline, the per-frame PCI-e transfer that bottlenecks the CPU path is eliminated. The benchmarks reflect this directly: the CPU pipeline drops below 60 FPS at 5,000 boids, while the GPU pipeline sustains real-time framerates past 1.5 million.

Using this pipeline as a case study shows that Unity's Compute Shader environment supports the same low-level parallel patterns covered in the labs. `InterlockedAdd` atomics handle race conditions in the grid construction phase, `groupshared` memory enables the on-chip Blelloch scan, and sorting boids by cell before the physics pass produces coalesced reads in `UpdateBoids`. The main tradeoff is the level of control over the project. CUDA offers a far more fine grained control which allows the user to optimize the performance of their application. However, Unity's UI and abstractions enables the programmer to move a project from the idea to the implementation phase much quicker as the code takes much less longer to write.